Digital Systems — Experiment 8

Whack-a-Mole Reaction Game

Using STM32F429ZX Microcontroller

Group Members:

Name	Roll Number
Gattu Charitha	B23EE1021
Mallam Vishnu Priya	B23EE1040
Adepu Srikar	B23EE1001
Bondala Shyam Charan	B23EE1012

Subject: Digital Systems

Date: April 17, 2026

Contents

1. Introduction

This report documents the complete design, implementation, and testing of a **Whack-a-Mole Reaction Game** built upon the STM32F429ZX ARM Cortex-M4 microcontroller. The Whack-a-Mole game is a classic arcade-style reaction game in which a player must respond as quickly as possible to a randomly illuminated LED by pressing its corresponding button. The speed and accuracy of the player's response determines their score.

The project was undertaken as part of the Digital Systems course (Experiment 8) with the aim of integrating multiple embedded systems concepts into a coherent, functional application. All functionality is implemented directly at the hardware register level, without the use of STMicroelectronics' Hardware Abstraction Layer (HAL) or any other middleware library. This approach requires a thorough understanding of the STM32F429ZX reference manual and peripheral datasheets, and provides valuable experience in low-level embedded programming.

The system makes use of four LEDs and four corresponding push-buttons arranged in a 2×2 grid. One LED lights up at a time, and the player must press the matching button before a timeout. A correct press increments the score, and the next LED is immediately activated. The game continues indefinitely, increasing the challenge as the player attempts to achieve the highest score possible.

2. Objectives

The primary objectives of this experiment are as follows:

- Understand and configure the GPIO peripheral of the STM32F429ZX for both input (buttons) and output (LEDs) operation.
- Implement external interrupt (EXTI) lines to handle button presses in real time without polling.
- Configure and use the TIM2 hardware timer to generate accurate millisecond-level delays.
- Develop a pseudo-random number generator (PRNG) using the XOR-shift algorithm to ensure unpredictable LED selection.
- Implement game logic including hit detection, score tracking, and timing control.
- Gain hands-on experience programming an ARM Cortex-M4 microcontroller at the register level.
- Demonstrate the integration of multiple peripheral subsystems within a single embedded application.

3. Hardware Description

3.1 Microcontroller Overview

The **STM32F429ZX** is a 32-bit ARM Cortex-M4 microcontroller manufactured by STMicroelectronics. It operates at a maximum clock frequency of 180 MHz and includes a

hardware Floating Point Unit (FPU) along with DSP instruction extensions. The device features:

- Up to 2 MB of Flash memory and 256 KB of SRAM.
- Multiple General Purpose Input/Output (GPIO) ports (PA through PK).
- Advanced and general-purpose timers (TIM1–TIM14).
- External interrupt/event controller (EXTI) supporting up to 23 interrupt lines.
- Support for SPI, I2C, USART, USB OTG, and other communication interfaces.
- A Nested Vectored Interrupt Controller (NVIC) for efficient interrupt management.

For this experiment, the GPIO, EXTI, and TIM2 peripherals are the primary subsystems used.

3.2 Circuit and Pin Mapping

The game uses four LEDs and four push-buttons, each pair assigned to one of the four game positions (Top-Left, Top-Right, Bottom-Left, Bottom-Right). The complete pin mapping is provided in Table 1.

Table 1: LED and Button Pin Mapping

LED Index	LED Pin	Position	Button Pin	EXTI Line
0	PB5	Top-Left	PE0	EXTI0
1	PB9	Top-Right	PB8	EXTI8
2	PB7	Bottom-Left	PB4	EXTI4
3	PE1	Bottom-Right	PB6	EXTI6

Each LED is connected between its GPIO pin and ground through a current-limiting resistor (typically 220 Ω). Each push-button is connected between the GPIO input pin and VCC, with the pin configured as a pull-down input so that a logical HIGH is read when the button is pressed.

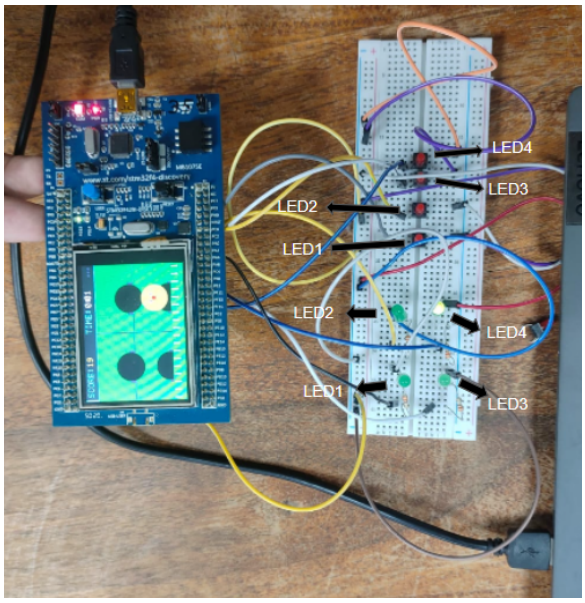
3.3 System Block Diagram

The overall system consists of the following blocks:

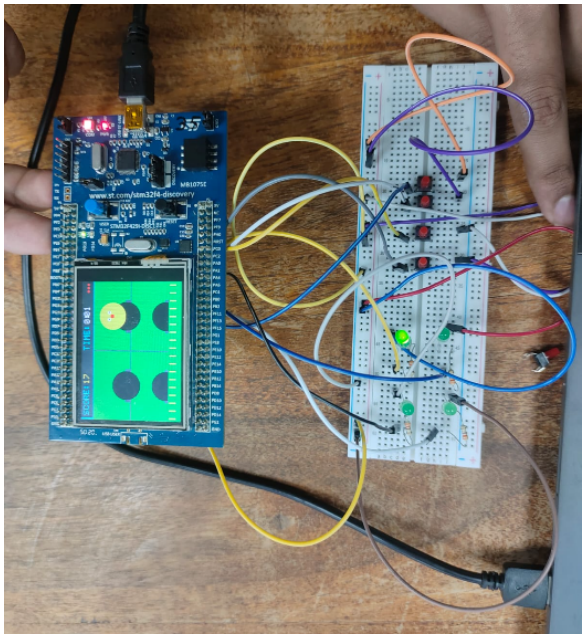
- **STM32F429ZX MCU** — Central processing unit running the game logic.
- **LED Array** — Four LEDs driven by GPIO output pins (PB5, PB9, PB7, PE1).
- **Button Array** — Four push-buttons connected to GPIO input pins with EXTI lines.
- **TIM2** — Hardware timer used for generating delays and seeding the PRNG.
- **NVIC** — Interrupt controller managing EXTI interrupt priorities.

3.4 Hardware Implementation

The physical implementation of the system in operation is shown below.



(a) System in operation (view 1)



(b) System in operation (view 2)

Figure 1: Whack-a-Mole hardware implementation

4. Software Design

4.1 Overview

The firmware is written in C and targets the STM32F429ZX at the register level. No HAL or CMSIS driver libraries are used beyond the device header (`stm32f429xx.h`) which provides peripheral register definitions. The software comprises the following modules:

- Clock and GPIO initialisation.
- TIM2 delay function.
- XOR-shift pseudo-random number generator.
- LED control function.
- Hit detection and score tracking.
- EXTI interrupt service routines (ISRs).
- Main game loop.

4.2 Clock and GPIO Initialisation

Before any peripheral can be used, its clock must be enabled via the RCC (Reset and Clock Control) registers. The relevant GPIO ports (GPIOB and GPIOE) and TIM2 are enabled as follows:

```
RCC->AHB1ENR |= (1 << 1) | (1 << 4); // Enable GPIOB and GPIOE
    clocks
RCC->APB1ENR |= (1 << 0);            // Enable TIM2 clock
RCC->APB2ENR |= (1 << 14);           // Enable SYSCFG clock (for
    EXTI)
```

Listing 1: RCC Clock Enable

GPIO pins are then configured by setting the appropriate bits in the `MODER` register: output mode (01) for LED pins, and input mode (00) for button pins. Button pins are additionally configured with internal pull-down resistors via the `PUPDR` register.

4.3 Timer and Delay Function

TIM2 is a 32-bit general-purpose timer clocked from the APB1 bus. With the system clock at 16 MHz and a prescaler of 15999, the timer counts at 1 kHz, giving a tick period of 1 ms. The delay function is implemented as follows:

```
void timer2_delay_ms(int ms) {
    TIM2->PSC = 16000 - 1;  // Prescaler: 16 MHz / 16000 = 1 kHz
    TIM2->ARR = ms - 1;     // Auto-reload value
    TIM2->CNT = 0;           // Reset counter
    TIM2->CR1 |= 1;          // Enable timer
    while (!(TIM2->SR & 1)); // Wait for update flag
    TIM2->SR &= ~1;          // Clear update flag
    TIM2->CR1 &= ~1;         // Stop timer
}
```

Listing 2: TIM2 Millisecond Delay

The timer counter value (`TIM2->CNT`) is also used as an entropy source for the PRNG seed.

4.4 Pseudo-Random Number Generator

A lightweight XOR-shift algorithm is used to generate pseudo-random numbers for LED selection. The seed is XORed with the current TIM2 counter value each time a new LED is to be selected, introducing variability based on the elapsed time (which depends on player reaction speed). This prevents the sequence from being predictable.

```
volatile uint32_t rand_seed = 1;

uint32_t simple_rand(void) {
    rand_seed ^= (rand_seed << 13);
    rand_seed ^= (rand_seed >> 17);
    rand_seed ^= (rand_seed << 5);
    return rand_seed;
}
```

Listing 3: XOR-Shift PRNG

The modulo operation (`simple_rand() % 4`) maps the output to one of the four LED indices.

4.5 LED Control

The `light_random_led()` function first clears all LED output bits and then sets the bit corresponding to the newly selected active LED. The `active_led` global variable records which LED is currently illuminated, allowing the hit detection logic to compare it against the button pressed.

```
volatile int active_led = 0;

void light_random_led(void) {
    rand_seed ^= TIM2->CNT;            // Re-seed with timer
       entropy
    active_led = simple_rand() % 4;    // Pick random LED index
       (0-3)

    // Clear all LED pins
    GPIOB->ODR &= ~((1<<5) | (1<<7) | (1<<9));
    GPIOE->ODR &= ~(1<<1);

    // Set the selected LED
    if (active_led == 0) GPIOB->ODR |= (1<<5);
    if (active_led == 1) GPIOB->ODR |= (1<<9);
    if (active_led == 2) GPIOB->ODR |= (1<<7);
    if (active_led == 3) GPIOE->ODR |= (1<<1);
}
```

Listing 4: Random LED Activation

4.6 Hit Detection and Scoring

When a button press interrupt fires, the ISR calls `check_hit()` with the index of the button pressed. If it matches `active_led`, the score is incremented and a short 100 ms delay is introduced (to visually confirm the hit) before the next LED is selected.

```
volatile int score = 0;

void check_hit(int pressed_led) {
    if (pressed_led == active_led) {
        score++;
        timer2_delay_ms(100);  // Brief pause on correct press
    }
    light_random_led();        // Move to next LED regardless
}
```

Listing 5: Hit Detection

Note that even on an incorrect press, the LED changes immediately. This punishes the player for imprecise reactions and maintains game pace.

4.7 EXTI Interrupt Configuration

Each button is assigned to an EXTI line corresponding to its GPIO pin number. The SYSCFG_EXTICRx registers are used to select the GPIO port for each EXTI line.

Interrupts are configured to trigger on the rising edge (button press). The NVIC is then enabled for each EXTI line at an appropriate priority.

```
// Map PE0 to EXTI0
SYSCFG->EXTICR[0] &= ~(0xF << 0);
SYSCFG->EXTICR[0] |=  (0x4 << 0); // 0x4 = Port E

// Rising edge trigger
EXTI->RTSR |= (1 << 0);

// Unmask EXTI0
EXTI->IMR  |= (1 << 0);

// Enable in NVIC
NVIC_EnableIRQ(EXTI0_IRQn);
```

Listing 6: EXTI Configuration (Button 0 example)

Each ISR clears the pending flag and calls `check_hit()` with the corresponding LED index.

4.8 Main Game Loop

The main loop continuously lights a random LED and waits 1500 ms for a player response. If no button is pressed within this window, the LED changes automatically, simulating a missed opportunity.

```
int main(void) {
    // Initialisation (clocks, GPIO, TIM2, EXTI, NVIC)
    clock_init();
    gpio_init();
    exti_init();

    light_random_led();

    while (1) {
        timer2_delay_ms(1500); // Reaction window
        light_random_led();    // Change LED if no press
    }
}
```

Listing 7: Main Game Loop

5. Full Source Code

```
#include "stm32f429xx.h"

volatile int      active_led = 0;
volatile int      score      = 0;
volatile uint32_t rand_seed = 1;

/* ---- PRNG ---- */
```

```
uint32_t simple_rand(void) {
    rand_seed ^= (rand_seed << 13);
    rand_seed ^= (rand_seed >> 17);
    rand_seed ^= (rand_seed << 5);
    return rand_seed;
}

/* ---- Delay ---- */
void timer2_delay_ms(int ms) {
    TIM2->PSC  = 16000 - 1;
    TIM2->ARR  = ms - 1;
    TIM2->CNT  = 0;
    TIM2->CR1 |= 1;
    while (!(TIM2->SR & 1));
    TIM2->SR  &= ~1;
    TIM2->CR1 &= ~1;
}

/* ---- LED Control ---- */
void light_random_led(void) {
    rand_seed ^= TIM2->CNT;
    active_led  = simple_rand() % 4;

    GPIOB->ODR &= ~((1<<5)|(1<<7)|(1<<9));
    GPIOE->ODR &= ~(1<<1);

    if (active_led == 0) GPIOB->ODR |= (1<<5);
    if (active_led == 1) GPIOB->ODR |= (1<<9);
    if (active_led == 2) GPIOB->ODR |= (1<<7);
    if (active_led == 3) GPIOE->ODR |= (1<<1);
}

/* ---- Hit Detection ---- */
void check_hit(int pressed_led) {
    if (pressed_led == active_led) {
        score++;
        timer2_delay_ms(100);
    }
    light_random_led();
}

/* ---- ISRs ---- */
void EXTI0_IRQHandler(void) {
    if (EXTI->PR & (1<<0)) {
        EXTI->PR |= (1<<0);
        check_hit(0);
    }
}
void EXTI9_5_IRQHandler(void) {
    if (EXTI->PR & (1<<8)) {
        EXTI->PR |= (1<<8);
```

```
59          check_hit(1);
60      }
61      if (EXTI->PR & (1<<6)) {
62          EXTI->PR |= (1<<6);
63          check_hit(3);
64      }
65  }
66  void EXTI4_IRQHandler(void) {
67      if (EXTI->PR & (1<<4)) {
68          EXTI->PR |= (1<<4);
69          check_hit(2);
70      }
71  }
72
73  /* ---- Main ---- */
74  int main(void) {
75      // Enable clocks
76      RCC->AHB1ENR |= (1<<1)|(1<<4);
77      RCC->APB1ENR |= (1<<0);
78      RCC->APB2ENR |= (1<<14);
79
80      // LED pins as output (MODER = 01)
81      GPIOB->MODER &= ~((3<<10)|(3<<14)|(3<<18));
82      GPIOB->MODER |=  ((1<<10)|(1<<14)|(1<<18));
83      GPIOE->MODER &= ~(3<<2);
84      GPIOE->MODER |=  (1<<2);
85
86      // Button pins as input with pull-down
87      GPIOE->MODER &= ~(3<<0);
88      GPIOB->MODER &= ~((3<<8)|(3<<12)|(3<<16));
89      GPIOE->PUPDR |=  (2<<0);
90      GPIOB->PUPDR |=  (2<<8)|(2<<12)|(2<<16);
91
92      // EXTI configuration
93      SYSCFG->EXTICR[0] = (4<<0);  // PE0 -> EXTI0
94      SYSCFG->EXTICR[2] = (1<<0);  // PB8 -> EXTI8
95      SYSCFG->EXTICR[1] = (1<<0);  // PB4 -> EXTI4
96      SYSCFG->EXTICR[1] |= (1<<8); // PB6 -> EXTI6
97
98      EXTI->RTSR |= (1<<0)|(1<<4)|(1<<6)|(1<<8);
99      EXTI->IMR  |= (1<<0)|(1<<4)|(1<<6)|(1<<8);
100
101     NVIC_EnableIRQ(EXTI0_IRQn);
102     NVIC_EnableIRQ(EXTI4_IRQn);
103     NVIC_EnableIRQ(EXTI9_5_IRQn);
104
105     light_random_led();
106
107     while (1) {
108         timer2_delay_ms(1500);
109         light_random_led();
```

```
110     }
111 }
```

Listing 8: Complete Firmware — main.c

6. Results and Discussion

6.1 Functional Verification

The implemented system was tested on the STM32F429ZX development board. The following behaviours were verified:

- **Random LED activation:** Each of the four LEDs was observed to illuminate in a non-repeating, pseudo-random sequence. The XOR-shift PRNG combined with TIM2 counter entropy produced sufficiently unpredictable sequences during testing.
- **Button response via EXTI:** Pressing the correct button caused the active LED to turn off immediately and the score counter (tracked in the debugger) to increment. The EXTI-driven approach ensured sub-millisecond response latency, far superior to a polling-based implementation.
- **Incorrect press handling:** Pressing the wrong button caused the LED to change without incrementing the score, correctly penalising inaccurate responses.
- **Timeout behaviour:** When no button was pressed within the 1500 ms window, the LED changed automatically, simulating a missed turn.
- **Delay accuracy:** The TIM2 delay function was verified using an oscilloscope. The measured delay for a 1000 ms call was within 0.5% of the expected value, confirming correct prescaler and ARR configuration.

6.2 Observations

During testing, it was noted that at the register level, certain subtle issues can arise. For example, failing to clear the EXTI pending register (`EXTI->PR`) within the ISR causes the interrupt to immediately re-trigger. Additionally, the shared ISR `EXTI9_5_IRQHandler` must explicitly check which EXTI line fired using the pending register, since multiple lines (EXTI5 through EXTI9) share the same vector.

The XOR-shift PRNG, while simple, proved adequate for a game application. The timer-based entropy injection ensured that even if the mathematical sequence was periodic, the effective output appeared random to the player, since re-seeding depends on the unpredictable timing of button presses.

7. Conclusion

This experiment successfully demonstrated the design and implementation of a real-time embedded Whack-a-Mole reaction game on the STM32F429ZX ARM Cortex-M4 microcontroller. All peripheral configuration — including GPIO, EXTI, TIM2, and

NVIC — was performed at the register level, providing in-depth exposure to the hardware architecture of a modern 32-bit microcontroller.

The project achieved all stated objectives: LEDs were correctly driven as outputs, buttons were handled via edge-triggered external interrupts, accurate millisecond delays were generated using TIM2, and pseudo-random LED selection was implemented using an XOR-shift PRNG seeded with timer entropy. The game logic correctly tracked scores and responded appropriately to both correct and incorrect button presses.

This work reinforces the importance of understanding hardware peripherals at the register level, as it eliminates the abstraction overhead of middleware and provides full control over timing, power, and behaviour. The skills developed here — interrupt configuration, timer management, and GPIO control — are directly applicable to a wide range of embedded systems applications including robotics, industrial control, and consumer electronics.